

```
task_id:[Loop2] [full_control-15] [INFO] [1617245621.881432804]
[tinyRobot_fleet_adapter]: Planning for [2] robot(s) and [1] request(s)
[full_control-15] [INFO] [1617245621.886230967] [tinyRobot_fleet_adapter]:
Submitted BidProposal to accommodate task [Loop2] by robot [tinyRobot2] with
new cost [45.222308]
```

[Click here](#) to learn how to develop support for a custom task.

Found a bug? [Edit this page on GitHub](#).

User-defined Custom Tasks in RMF Task

Note: User-defined custom tasks are currently experimental

A new general task composition system is under progress the discussion can be found [here](#).

When dealing with RMF tasks, there are two packages:

- [rmf_task](#)
- [rmf_task_sequence](#)

`rmf_task` provides APIs and base classes for defining and managing Tasks in RMF. A Task is defined as an object that generates phases which are a meaningful sequence of steps that results in a desirable outcome. Each task will have a description and a component that allows us to model the state of the robot after completing the task given its initial state and also a component that will command the actual robot to perform the task.

`rmf_task_sequence` provides an out of the box implementation of `rmf_task` where a `Task` object is defined by a sequence of phases. The phases that such tasks generate will thus match the sequence of phases used to define them. The phases defined in `rmf_task_sequence` are in turn a collection of events which also have components to model the end state and command the robot during the event. Presently the events defined [here](#) are supported. Users can then construct arbitrary definitions of tasks by stringing together a sequence of such phases/events. RMF is capable of planning and executing such tasks.

`perform_action` is a sequence based event which supports performing custom actions. The customizability of the behavior to `perform_action` is limited so that users who are implementing custom logic don't need to worry about how to interact with the traffic system, or open doors, or use lifts. This also minimizes the risk of the system integrator introducing an error that messes up the traffic system or any other resource-sharing systems. When `perform_action` is running, the robot becomes a "read-only" traffic agent, so other robots will simply avoid it

Users will be working on the `rmf_fleet_adapter` layer. In this layer the API is restricted to use only `rmf_task_sequence` to perform tasks. Only certain events are supported, their description can be found [here](#).

The `rmf_fleet_adapter` layer acts as an API that users can use. It supports customised behaviour on only `perform_action` in the already existing events mentioned [here](#). Users can only add custom tasks in `perform_action`. RMF passes the command to the platform-specific end of the fleet adapter integration and formally releases control of the robot until the action is finished. To use a custom task in `perform_action` users need to use two parts of the API.

1. `FleetUpdateHandle::add_performable_action` This consists of two parts: the first one is the `category` of the action and the second is the `consider` part, based on which it would be decided whether to accept the action or not.

Here is an example:

```
rmf_fleet:
  name: "ecobot40"
  limits:
    linear: [1.2, 1.5] # velocity, acceleration
    angular: [0.6, 0.6]
  profile: # Robot profile is modelled as a circle
    footprint: 0.5
    vicinity: 0.6
  reversible: False
  battery_system:
    voltage: 24.0
    capacity: 40.0
    charging_current: 26.4
  mechanical_system:
    mass: 80.0
    moment_of_inertia: 20.0
    friction_coefficient: 0.20
  ambient_system:
    power: 20.0
  cleaning_system:
    power: 760.0
  recharge_threshold: 0.05
  recharge_soc: 1.0
  publish_fleet_state: True
  account_for_battery_drain: True
  task_capabilities: # Specify the types of RMF Tasks that robots in this fleet
are capable of performing
    loop: True
    delivery: False
    clean: True
    finishing_request: "park"
    action_categories: ["clean", "manual_control"]
```

```

def _consider(description: dict):
    confirm = adpt.fleet_update_handle.Confirmation()

    # Currently there's no way for user to submit a robot_task_request
    # .json file via the rmf-web dashboard. Thus the short term solution
    # is to add the fleet_name info into action description. NOTE
    # only matching fleet_name action will get accepted
    if (description["category"] == "manual_control" and
        description["description"]["fleet_name"] != fleet_name):
        return confirm

    node.get_logger().warn(
        f"Accepting action: {description} ")
    confirm.accept()
    return confirm

fleet_config = config_yaml['rmf_fleet']
task_capabilities_config = fleet_config['task_capabilities']
if 'action_categories' in task_capabilities_config:

    for cat in task_capabilities_config['action_categories']:

        node.get_logger().info(

            f"Fleet [{fleet_name}] is configured"

            f" to perform action of category [{cat}]")

        fleet_handle.add_performable_action(cat, _consider)

```

2. [RobotUpdateHandle::set_action_executor](#) This is where you tell the fleet adapter how to instruct your robot to begin performing an action. The callback on this function consists of:

- `category(string)` type of action.
- `description(JSON)` message which contains details about how the action should be performed.
- `execution(object)` object which the platform-specific side of the fleet adapter must hold onto while the action is being performed, ideally giving periodic updates for remaining time estimates.

The robot will not participate in the traffic negotiations while using a custom task in `perform_action`. That means that it will be allowed to report its trajectory to the traffic schedule, thus making it possible for other robots to avoid it. However, the robot would not be able to accommodate other robots until the task is complete.

Here is an example:

```

# call this when starting cleaning execution
def _action_executor(self,
                      category: str,
                      description: dict,
                      execution:
                        adpt.robot_update_handle.ActionExecution):
    with self._lock:
        # only accept clean and manual_control
        assert(category in ["clean", "manual_control"])

        self.action_category = category
        if (category == "clean"):
            attempts = 0
            self.api.set_cleaning_mode(self.config['active_cleaning_config'])
            while True:
                self.node.get_logger().info(f"Requesting robot {self.name} to
clean {description}")
                if self.api.start_clean(description["clean_task_name"],
self.robot_map_name):
                    self.check_task_completion = self.api.task_completed #
will check api
                    break
                if (attempts > 3):
                    self.node.get_logger().warn(
                        f"Failed to initiate cleaning action for robot
[{self.name}]")
                    # TODO: issue error ticket
                    self.in_error = True # TODO: toggle error back
                    execution.error("Failed to initiate cleaning action for
robot {self.name}")
                    execution.finished()
                    return
                attempts+=1
                time.sleep(1.0)
            elif (category == "manual_control"):
                self.check_task_completion = lambda:False # user can only cancel
the manual_control

        # start Perform Action
        self.node.get_logger().warn(f"Robot [{self.name}] starts [{category}]
action")

        self.start_action_time = self.adapter.now()
        self.on_waypoint = None
        self.on_lane = None
        self.action_execution = execution
        self.stubbornness = self.update_handle.unstable_be_stubborn()
        # robot moves slower during perform action
        self.vehicle_traits.linear.nominal_velocity *= 0.2

    self.update_handle.set_action_executor(self._action_executor)

```

These examples are part of the following [repository](#).

Found a bug? [Edit this page on GitHub](#).

Supporting a new Task in RMF

With the release of [RMF Task V2](#), users can now construct custom tasks according to their specific needs. Different combination or sequence of robotic tasks can be dispatched to a specified robot or to the best available fleet based on the users' preferences.

The new flexible task system introduces the concept of a Phase. A task is an object that generates phases. In other words, a task is typically made up of a series or combination of phases as its building blocks. For example, a delivery task would require a robot to complete the following steps:

1. Move from its current waypoint to a pick-up location
2. Pick up the delivery payload
3. Move from the pick up location to the drop-off location
4. Drop off the payload
5. Move back to the initial starting waypoint

Each of these steps can be considered a Phase. Users can use the following public API phases to construct their own tasks:

- [GoToPlace](#)
- [PickUp](#)
- [DropOff](#)
- [PerformAction](#)

Additional phase descriptions, including those supporting the public API phases, are defined and listed [here](#). They will be useful for building your own custom task.

Certain tasks may require specific phases that are not mentioned above. For example, if a delivery task involves the robot moving from the first to second level, it would require a `RequestLift` phase. Such phases are used by RMF internally and automatically added to a task when necessary, so users do not need to worry about them when creating their custom tasks.

Building a Custom Task

Users can build and send their own tasks by publishing [ApiRequest](#) messages. You will need to fill in the `request_id` and `json_msg` fields according to the types of phases that make up the task, as well as whether the task is intended for a specific robot or the best available fleet. You may follow these steps to construct your own task:

1. Create an `ApiRequest` publisher that sends task requests via the `/task_api_requests` topic.
2. Fill in the `request_id` field with a unique string ID that can be used to identify the task.
3. For the `json_msg` field,
 - Use the `robot_task_request` schema and fill in the JSON payload type with `"robot_task_request"` to send a task request to a specific robot
 - Use the `dispatch_task_request` schema and fill in the JSON payload type with `"dispatch_task_request"` to send a task request to the best available fleet
 - The `request` fields for these objects follow the `task_request` schema
4. Populate the object fields with the required information.
 - The `category` and `description` fields under the `task_request` schema take in the string name of the task and the task description respectively. The JSON schema for these descriptions can be found [here](#). There are currently four task descriptions available:
 - **Clean**: create your own clean task, requires the `Clean` phase description
 - **Compose**: create your own custom task that may comprise of a sequence of phases, requires descriptions for the relevant phases
 - **Delivery**: create your own delivery task, requires the `PickUp` and `DropOff` phase descriptions
 - **Patrol**: create your own patrol task, requires the `Place` description to indicate where you would like your robot to go to
5. Publish the `ApiRequest` !

Examples of JSON Task Requests

For a **Clean** `dispatch_task_request` :

```
{
  "type": "dispatch_task_request",
  "request": {
    "unix_millis_earliest_start_time": start_time,
    "category": "clean",
    "description": {
      "zone": "clean_lobby"
    }
  }
}
```

For a **Compose** `robot_task_request` that commands a specific robot to go to a place, followed by performing a `teleop` action:


```

{
  "type": "robot_task_request",
  "robot": "tinyRobot1",
  "fleet": "tinyRobot",
  "request": {
    "category": "compose",
    "description": {
      "category": "teleop",
      "phases": [
        {"activity": {
          "category": "sequence",
          "description": {
            "activities": [
              {"category": "go_to_place",
               "description": "coe"
            },
            {"category": "perform_action",
             "description": {"category": "teleop", "description": "coe"}}
          ]
        }
      ]
    }
  ]
}

```

For a **Delivery** dispatch_task_request:

```
{
  "type": "dispatch_task_request",
  "request": {
    "category": "delivery",
    "description": {
      "pickup": {
        "place": "pantry",
        "handler": "coke_dispenser",
        "payload": [
          {"sku": "coke",
            "quantity": 1}
        ]
      },
      "dropoff": {
        "place": "hardware_2",
        "handler": "coke_ingestor",
        "payload": [
          {"sku": "coke",
            "quantity": 1}
        ]
      }
    }
  }
}
```

For a **Patrol** `robot_task_request`:

```
{
  "type": "robot_task_request",
  "robot": "tinyRobot1",
  "fleet": "tinyRobot",
  "request": {
    "category": "patrol",
    "description": {
      "places": ["pantry", "lounge"],
      "rounds": 2
    }
  }
}
```

Some examples of composed task requests can be found [here](#) as reference. They can be used with `rmf_demos`. Feel free to modify these files according to your own application.

Task Management Control

You may take additional control over your tasks by sending requests to RMF to cancel a task or skip a phase. A full list of JSON schemas for such requests are defined [here](#).